

Ctrl+ArcZ

Screened before it is signed. Recallable until it is claimed. Repeatable without handing over your wallet.

USDC payments on Arc with the three things a plain transfer does not have: a firewall that refuses a bad recipient before anything is signed, a lock the sender can undo until the recipient proves the money was meant for them, and a bounded spend box that lets a merchant or an agent charge you again without ever touching your wallet.

ARC TESTNET · LIVE

CHAIN ID 5042002

0x8dAb7148cdc31DAcad6d7e12161AA3DEDb572Dca

	ANDROID	play.google.com/store/apps/details?id=com.xyz.ctrlarcz
	WEB	ctrlarcz.xyz
	SOURCE	github.com/Farukest/Ctrl-ArcZ
	SDK	npmjs.com/package/@ctrl-arcz/sdk
	IOS	Coming soon

THE DIFFERENCE

What we brought to Arc



Firewall, two layers

Every send is screened for poisoning, lookalike addresses and zero-value spoofing by a deterministic policy engine that fails closed, then by an agentic layer, an actual model, that weighs the whole on-chain picture together.

✧ What we brought to Arc

A second, agentic tier clamped so it can only ever tighten the rules, putting model-grade judgment in front of Arc payments without ever being able to approve a bad one.



Undoable Send

The receiver claims with a secret code you hand a person, so they do not even need a wallet, and until they claim the funds stay escrowed, never credited to anyone.

✧ What we brought to Arc

A human-claimable escrow layer where money sits in the contract as honestly pending, so no one can flash a “payment sent” and reverse it later, plus an autonomous keeper agent that auto-refunds expired transfers from an on-chain policy-capped box, so even the refund path is trustless.



Private Pay

Payments are routed so the link between sender and receiver never lands on chain.

✧ What we brought to Arc

Stealth-address communication and fresh disposable payer boxes, with an enclave co-signer (“The Machine”) holding the key and releasing only after policy and the firewall both pass, bringing unlinkable, TEE-gated spending to Arc.

THE DIFFERENCE

What we brought to Arc



Subscription

Recurring pulls from a spend box, every pull approved by the co-signer within an on-chain policy, capped and revocable, without exposing your main wallet.

✧ What we brought to Arc

Used Gateway unified balance to carry privacy and cross-chain funding onto Arc, so a subscription can be funded across chains and charged on schedule while the payer's main wallet stays unseen.



Disposable, one-time boxes

Every private payment and subscription runs through a fresh account, never your own wallet.

✧ What we brought to Arc

A CREATE2 policy account spun up behind an ephemeral stealth key for each payment, funded, used once, then swept and abandoned, so no address is reused and no standing approval is ever left behind.



Policy enforced by the contract

The money leaves from the box, and the box carries its own limits.

✧ What we brought to Arc

Target lock, per-pull cap, interval, total budget and expiry all live on chain and are enforced by the contract itself, so a misbehaving agent or a stolen co-signer key still cannot spend a cent beyond what the operator set.

THE DIFFERENCE

What we brought to Circle



Turned CCTP and Gateway unified balance from a pure bridge into a privacy and funding primitive: disposable boxes and subscriptions are funded and charged across chains without the payer's main wallet ever being exposed.



Gasless claim through Circle Modular Wallets, so a receiver holding zero gas can still claim an escrowed transfer, sponsored by a smart account and bundler.



Crash-safe cross-chain spends, where a transfer interrupted mid-flight is finished from its transferId alone, idempotent and with no double mint.



Put an enclave co-signer and on-chain policy caps on top of Gateway pulls, so a unified-balance payment becomes recurring, capped and revocable instead of one-shot.

THE PROBLEM

A plain transfer is final, blind and one-shot

Final

Signed is settled. There is no undo on a plain transfer, for the right recipient or the wrong one.

Blind

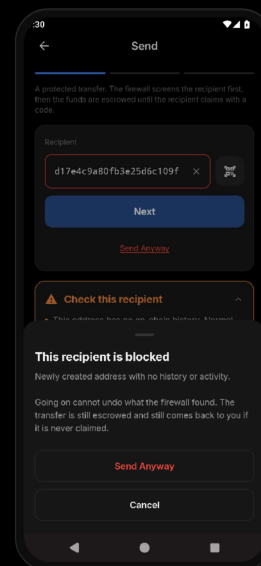
Nobody reads the middle of an address. Poisoning lives here: the attacker grinds a lookalike of one you already paid, and the **\$1 test transfer** confirms perfectly, to the wrong one too.

0x64Ea...Fe3F

the one you pay

0x64Ea...Fe3F

the lookalike the attacker ground out



A block is a gate, not a colour: Send Anyway opens a sheet that names the reason, with Cancel under it.

One-shot

A payment that must repeat rests on an unlimited approve or a shared key: the merchant, or the agent, is handed more than the payment.

THE SOLUTION

Four layers, one contract, no custody

 Layer 1

Firewall

A graded verdict before anything is signed.

sendProtected runs the scan itself, so installing the SDK is what makes a send protected.

 Layer 2

Protected transfer

USDC sits in the contract until the recipient proves a code. The sender can cancel; expiry refunds automatically.

 Layer 3

Clean history

Zero-value bait and unknown tokens drop out of the list, and every settled claim feeds back into Layer 1.

 Layer 4

Payer shield

The paying side is shielded too: a spend box owned by a fresh **stealth address**, a co-signer on every draw, and an agent key bounded by the contract.

Not another wallet. One deployment, many tenants: an integrator registers a config and gets its own recall window, claim mode and minimum worth protecting.

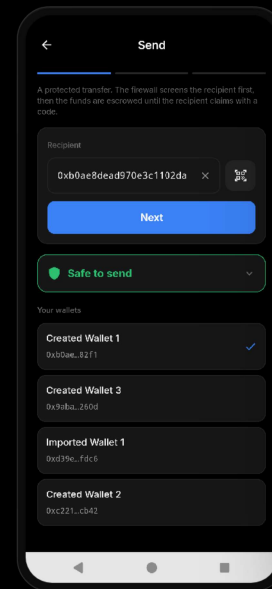
LAYER 1 IN THE FLOW

The send, split into decisions

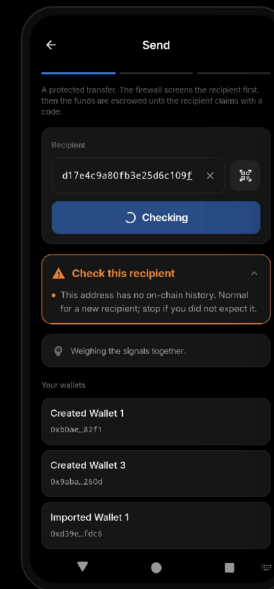
- 01 Step one is only the recipient. The firewall screens it **before the amount exists**.
- 02 This one reads **safe** because a transfer here already settled with a code, Layer 3 feeding Layer 1.
- 03 Two answers, not one. The rule engine decides first; the investigator can then **tighten** that verdict and can never lift it.

Safe to send

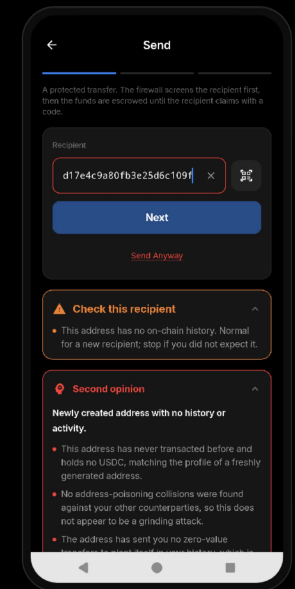
A protected transfer to this address settled before, claimed with a code.



Recipient first, and the verdict is a pill you can open.



Weighing the signals together, while the button reads Checking.



Amber from the rules, red from the investigator: the clamp working in one picture.

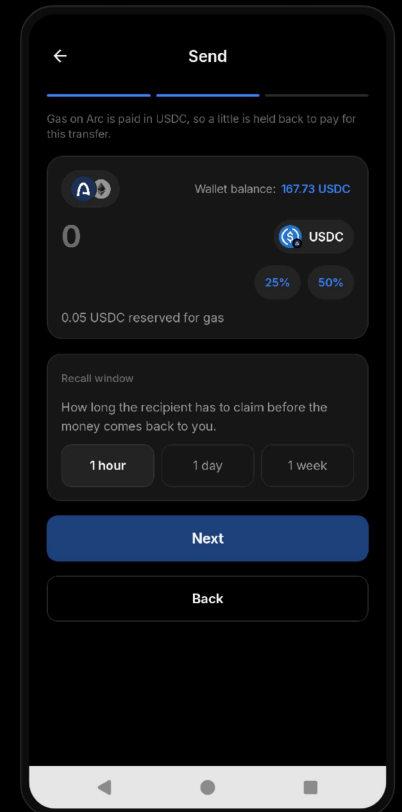
LAYER 2 IN THE FLOW

Amount and gas on Arc

On Arc, gas is paid in USDC, so the amount step has to be honest about it.

Reserved for gas	0.05 USDC
Recall window	1 hour · 1 day · 1 week
If unclaimed	returns to the sender

Confirm restates the recipient, the verdict and the amount while nothing has moved yet.



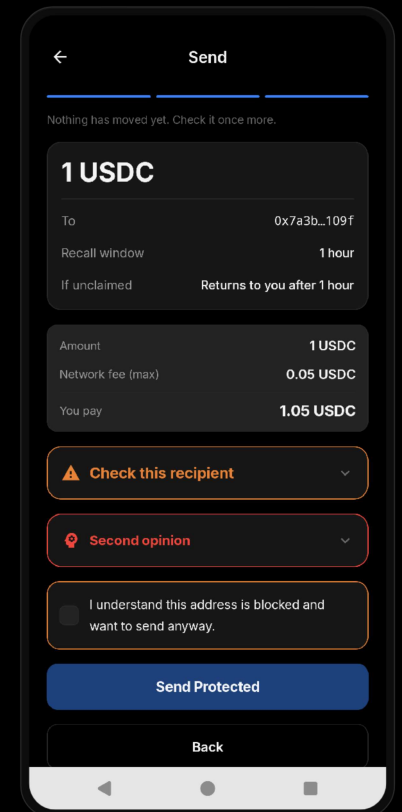
The balance is the way to take all of it, and what gas holds back is named under the field.

CONFIRM

Every screen that spends says what it will cost

On Arc, gas is USDC too. An amount without its fee is a lie.

- 01 Confirm ends in three lines: **Amount**, **Network fee (max)**, and under the rule the one number that is true: **You pay**.
- 02 The same block closes a send, a private payment and a bridge; a subscription states its own, total, end date and balance, before it starts charging.
- 03 Off Arc the block changes shape: the amount in USDC, the fee in the chain's own coin, and **no total**, because adding them would invent an exchange rate.



1 plus 0.05 is 1.05: amount, fee, then the number that is true.

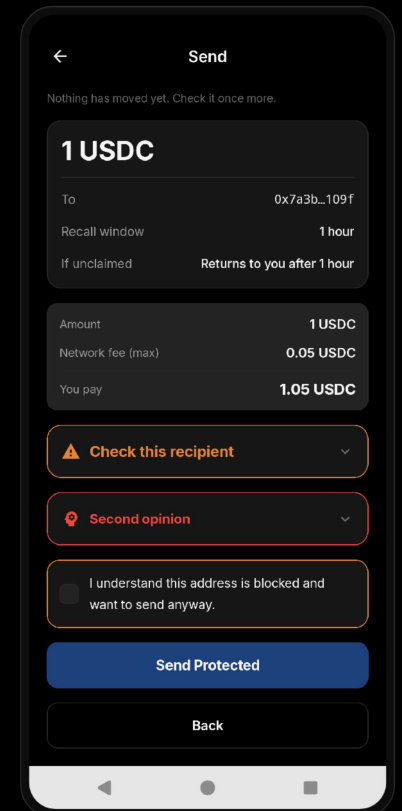
GRADED VERDICTS

The warning gate

- 01 An address with no on-chain history gets a **warning**, not a block.
- 02 Send Protected stays disabled until the acknowledgement is ticked by hand, and the wording follows the verdict: “I checked this recipient and want to send anyway.” for a warning, “I understand this address is blocked and want to send anyway.” for a block.
- 03 A positive signal never overrides a block, and if the data source is down the firewall **fails closed**.

Check this recipient

This address has no on-chain history. Normal for a new recipient; stop if you did not expect it.



The cost block, both verdicts collapsed, and the acknowledgement in the verdict's own words.

LAYER 1, EXTENDED

An investigator that can only tighten

POST /api/investigate gathers the signals the rule engine cannot combine. Its answer moves a verdict in one direction only.

- 01 Every reply is clamped to the rule engine's decision with **max**: a wrong or prompt-injected answer can refuse a good payment — it cannot approve a bad one, and it cannot clear a lookalike.
- 02 No key, a timeout, a malformed reply: the app returns to its **rule-only self**. The feature disappears; nothing else degrades.



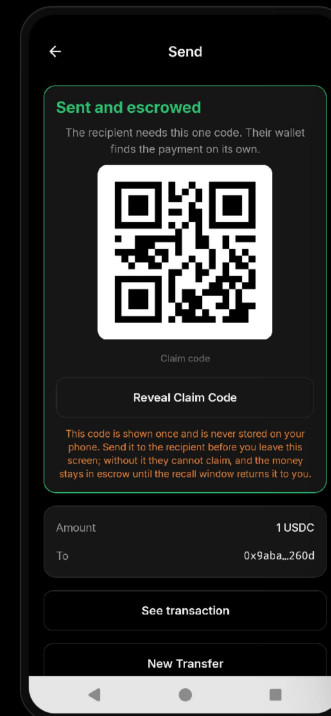
A `SAFE` answer never lifts anything.

THE SECOND FACTOR

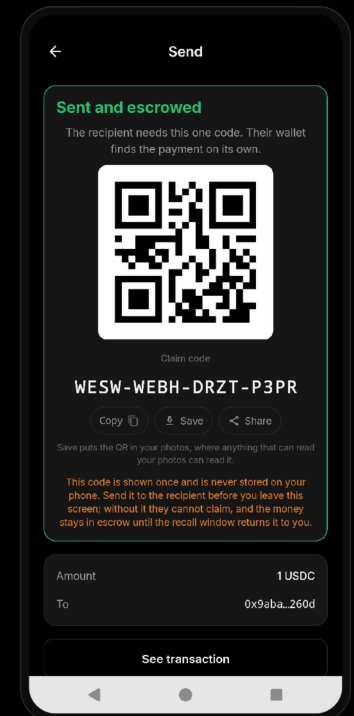
One code, handed over once

Q6MX 9CD8 PZ40 6DG2

- 01 Sixteen characters, **80 bits**, Crockford base32. It has to survive an offline brute force, because the recorded recipient may be the attacker.
- 02 No link, no transfer id, no salt travels with it. The chain only ever sees the hash.
- 03 The reveal screen sets **FLAG_SECURE** in the released build: no screenshot, no recording.
- 04 Under the code: **Copy, Save, Share**, and Save says it plainly: “Save puts the QR in your photos, where anything that can read your photos can read it.”



Money is in the contract; the code stays masked.



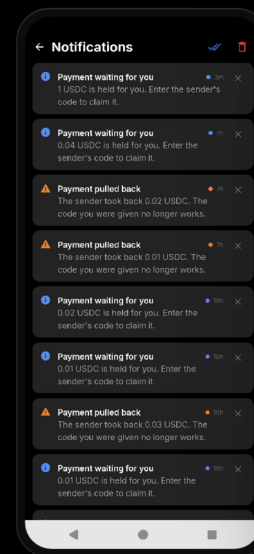
Revealed deliberately, once, to a person. One 1 USDC send: the amount under the QR and the amount in the summary agree.

THE OTHER SIDE

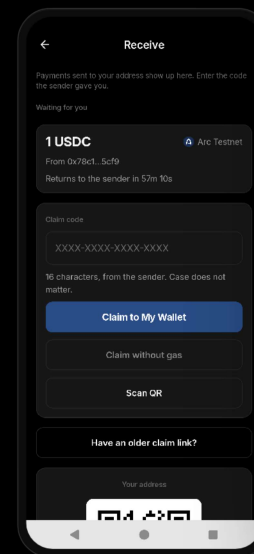
Notifications without a server

No watcher service. The app reads the contract's events straight from the chain: every 15 seconds while it is open, and on a WorkManager job when it is not. Notifications are raised on the device.

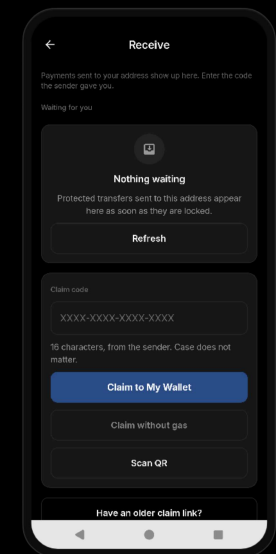
- 01 Filter is **TransferCreated** with the third indexed topic `to = you`. No push service, no Expo, nothing to register with.
- 02 Two tempos: a **15 second loop** while the app is open, and a background job when it is not: one shot on wallet connect, then every 15 minutes. The platform will not schedule that job more often, which is why the foreground loop exists.
- 03 Delivery is **idempotent**. The cursor lives on the device and does not advance for an event it could not deliver, so the event is caught on the next pass.
- 04 Written twice: a persistent in-app record, plus a system banner.
- 05 Scope, stated plainly: **protected transfers only**. Incoming Sends, and claim, expire or recall on the ones you sent. Private Pay and Gateway arrivals are ordinary ERC-20 movements and raise nothing.



Raised on the device, from the event.



A waiting payment names its chain: 1 USDC on Arc Testnet, returning to the sender in 57m 10s.



Nothing waiting, which is what a new recipient sees first.

DISCOVERY

Discovery is served, not delivered

GET /api/announcements

— same bytes, every caller —

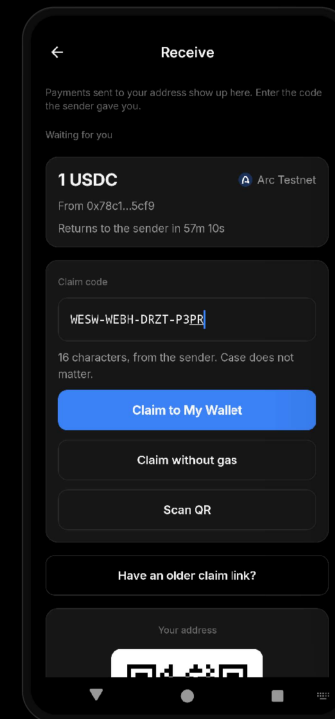
viewing key · in your browser

- 01 The endpoint takes **no address**. Which announcement is yours is decided on your device, with the viewing key, it is never asked, so it is never known.
- 02 The notification is a convenience, not the mechanism: Receive **reads the chain itself**, so a pending transfer shows up even with notifications off.
- 03 **One request** where there used to be a 219-part one, and neither the server nor an explorer learns who asked.

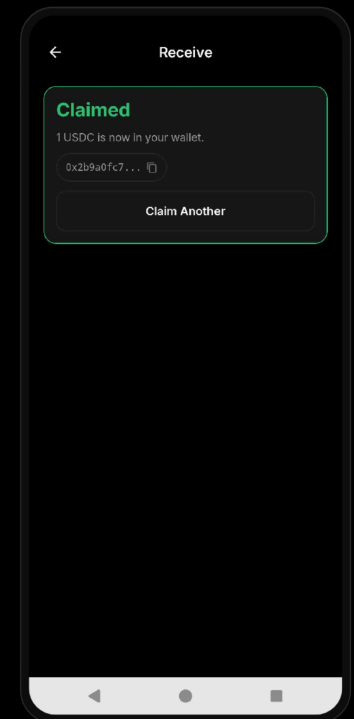
SETTLEMENT

Claim without gas

- 01 No link is sent. The recipient's wallet finds the pending payment **from its own address**.
- 02 A wallet holding **no USDC at all** claims anyway: the claim rides an **EntryPoint v0.7** user operation and Circle Gas Station's paymaster pays the gas. The relayer's balance does not move.
- 03 Measured: a Circle Smart Account holding nothing claimed; the paymaster paid **0.0062 USDC**, the relayer's delta was **0**, and the recipient's nonce stayed 0.
- 04 Cancel is the sender's at any time; after the window, **reclaimExpired** refunds, only ever to the sender.



The code typed in, against the waiting card it belongs to.



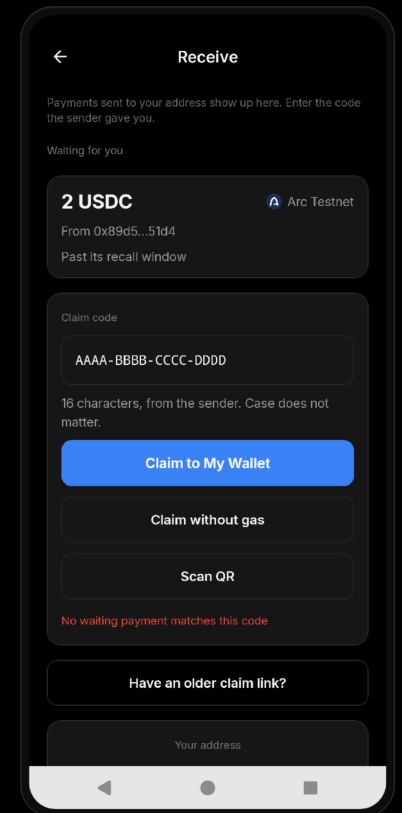
Claimed: 1 USDC is now in that wallet, with the hash. The wallet paid no gas to get it.

CONTRACT BEHAVIOUR

Wrong code, no cost

A wrong code returns false. It does not revert.

- 01 An attempt limiter cannot be built on a reverting call, because the revert would roll back the counter recording the guess.
- 02 Five wrong guesses freeze the transfer; the sender can still cancel and re-send.
- 03 A mistyped code in the app **burns no on-chain attempt**. The five stay for a real attacker.

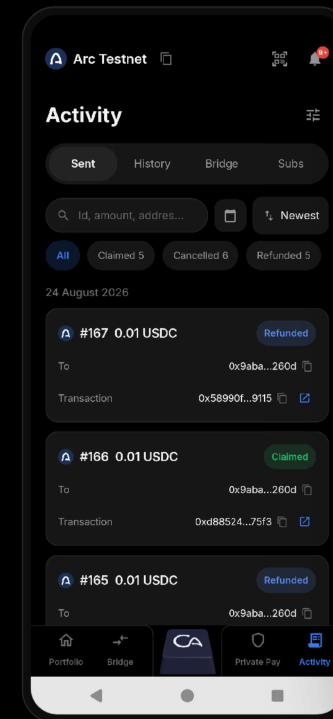


Rejected in the app, attempts untouched.

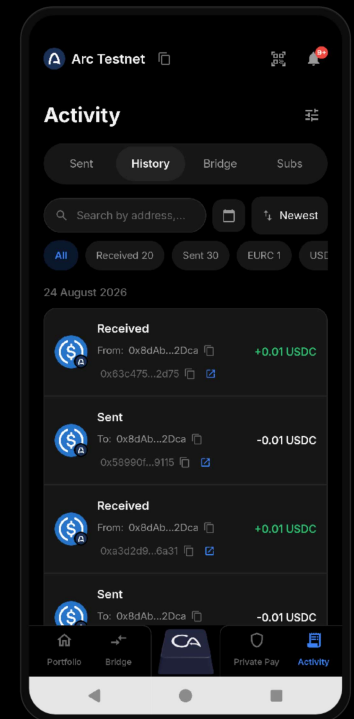
LAYER 3

History that stays readable

- 01 Zero-value bait transfers and unknown tokens are **filtered out of the list**, so the poisoned address never enters your history to be copied later.
- 02 Activity keeps the claim code with the transfer it belongs to, so a sender can hand it over again, and searches by **id, amount, address start or end, or code**.
- 03 Every settled claim becomes a positive signal for the firewall on the next send.



Pending, refunded, claimed, each with its code and hash.



Which chain, on every row.

FOR INTEGRATORS

A few calls, and the firewall runs inside them

You cannot forget to call the firewall. **sendProtected** runs it before it will sign anything.

- 01 A blocked recipient throws **RiskBlockedError** with the report and its reasons attached, so the integrator renders the verdict instead of inventing one.
- 02 **186 exported names** live at runtime in `@ctrl-arcz/sdk`: 122 functions and 64 values: `send`, `claim`, `cancel`, `reclaim`, `scan`, `subscriptions`, `bridge`.
- 03 **388 unit tests** on the SDK alone; **691** across the repo: 124 Foundry, 388 SDK, 146 demo-kit, 33 keeper.

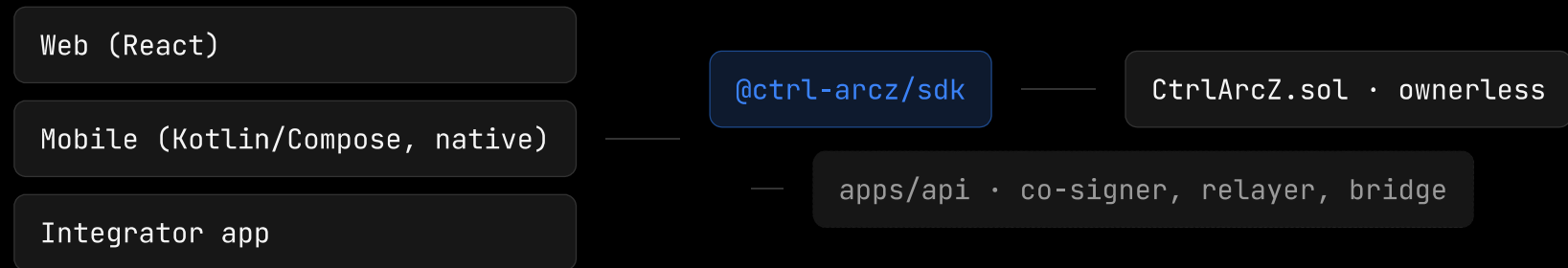
```
import { defineConfig, registerConfig,
  generateClaimCode, approveUsdc,
  sendProtected, RiskBlockedError }
  from '@ctrl-arcz/sdk'

const { configId } = await registerConfig(
  clients, defineConfig({ recallWindow: 3600 }))
const secret = generateClaimCode()
await approveUsdc(clients, amount)

try {
  const { transferId } = await sendProtected(
    clients,
    { configId, to, amount, claimHash: secret.claimHash },
    { config },
  )
} catch (err) {
  if (err instanceof RiskBlockedError)
    showVerdict(err.report)
}
```

ARCHITECTURE

Three clients, one SDK, one contract



Android is native Kotlin/Compose against the same `apps/api` and the same contract. What keeps the two apps in line is `packages/sdk/parity-vectors.json`: both test suites run against it, so a port that drifts fails a test, not a payment.

The contract holds the rules and the money. The API only pays gas, co-signs and relays; it cannot move a locked transfer, and neither can we.

THE WEB CLIENT

The same rules, rendered in a browser

Recipient address

0x64ea27bc67469aa7ef9dfaf8dedd42db5de0fe3f

Try the poisoning attack

Crafted to imitate 0x64Ea...Fe3F. Both render the same in a wallet.

Send blocked

Rules 2 findings

This address has no on-chain history. Normal for a new recipient; stop if you did not expect it.

The risk check was incomplete (verified recipients (partial scan) did not respond). We will not say "safe" without a full scan.

Agent check Could not be reached, rules only

[Send without verifying this address](#)

Amount

0.02

Balance 176.709429 USDC

\$0.02

Below the threshold, an unprotected plain transfer may be cheaper.

Send blocked, two reasons, and the override is a sentence you must mean.

New subscription

Merchant

Netflix

Merchant address

0x...

Per pull

0.02

\$0.02

USDC

How often

Every minute

How many minutes

6

Your every minute payment total

0.12 USDC

Circle fee

0.03956 USDC

You pay

0.15956 USDC

Create subscription

New subscription: Netflix picked with its logo, the address label amber, You pay 0.15956.

Gateway balance

3.687096 USDC

Arc Testnet

Wallet balance 176.699429 USDC

0.00

Deposit

Depositing on Arc Testnet takes about 1s to count.

From Arc Testnet

Amount

1

\$1.00

Gateway balance 3.687096 USDC

USDC

25% 50%

To Base Sepolia

You receive

1

\$1.00

Balance

USDC

Circle fee

0.112786 USDC

You pay

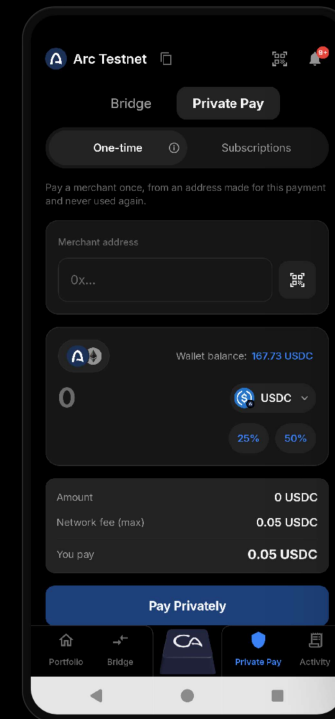
1.112786 USDC

Gateway, Arc Testnet to Base Sepolia: Circle fee 0.112786, You pay 1.112786.

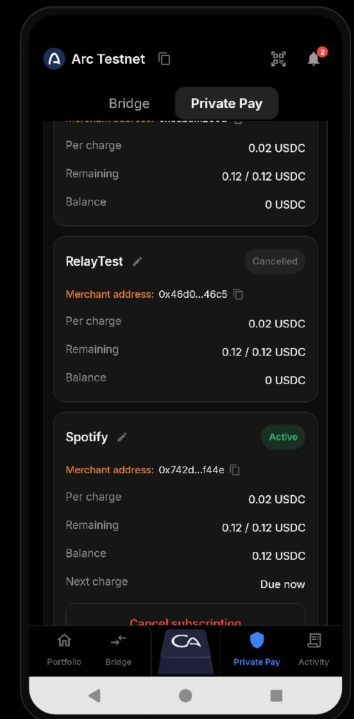
BEYOND ONE PAYMENT

Subscriptions and spend boxes

- 01 A one-time box pays a merchant without your address reaching them. The box is owned by a fresh **ERC-5564** stealth address.
- 02 The same box in **PULL mode** is a subscription: cap per pull, interval, total budget and remaining, all read from chain.
- 03 An **agent wallet** is the same box with a machine holding the key: it may draw up to the cap, on the interval, until the budget runs out, and the owner cancels in one transaction. The limit is enforced by the contract, not by the agent's good behaviour.



A one-time box: chain chip, wallet balance, and the cost block above Pay Privately.

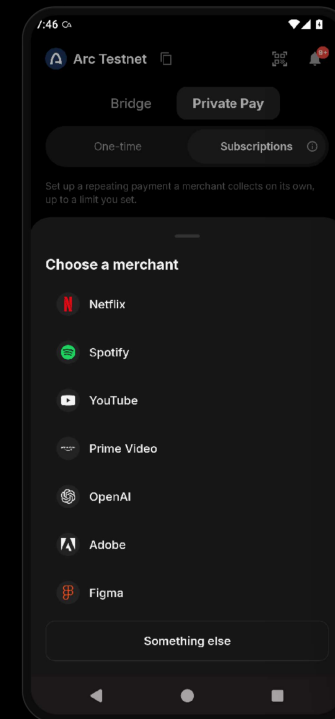


Merchant by name; per charge, remaining and balance from chain.

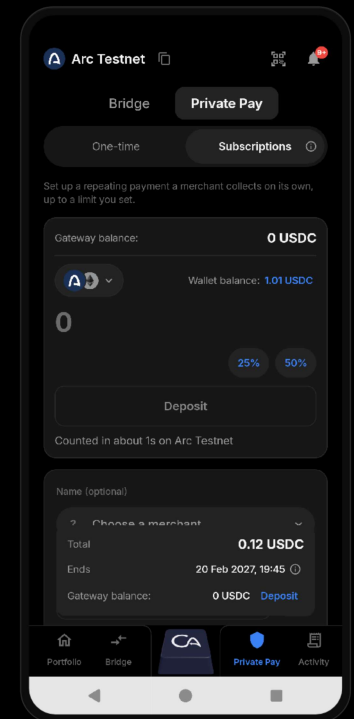
SUBSCRIPTIONS

The merchant is picked, not typed

- 01 Not cosmetic: the name is **packed into the box's stealth announcement**, so a typo would follow the subscription onto every device that reads it.
- 02 Netflix, Spotify, OpenAI and the rest sit in the picker; **Something else** remains for the merchant we did not list.
- 03 On the web, the address label turns **amber**: it is the one field the firewall judges.



Picked from a list;
“Something else” for
the rest.



Logo and name in the pill,
the summary underneath.

LAYER 4, PROVEN

An agent bounded by the chain

The keeper runs unattended, holding its own real key. What bounds it is the contract, not its judgement.

- 01 It reclaimed **#50** for real. The event names the keeper as `caller` and the original sender as `sender`; the recorded recipient received nothing.
- 02 Then it attacked its own budget, four ways. The contract refused all four.

```
TransferReclaimed(#50)
```

```
caller → the keeper's own key
```

```
sender → the original sender, refunded
```

Draw ten times the cap

Refused

Forge the co-signer's signature

Refused

Sweep the box to itself

Refused

Sweep to the correct vault

Refused

PRIVACY TODAY

Five steps on chain, and none of them names the payer

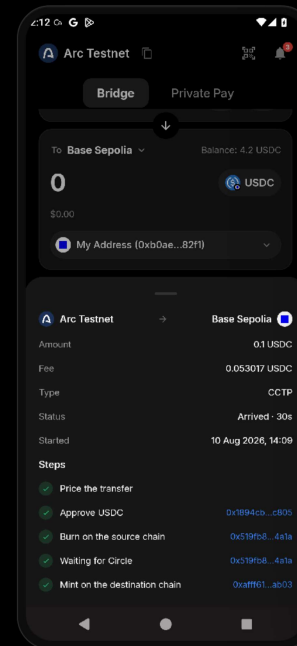
STEP	WHO SENDS IT ON CHAIN	LINKS TO THE PAYER
Deploy the box	Relayer	No
Announce the stealth address	Relayer	No
Fund the box	Circle's minter · Gateway mint	No
Gas for the stealth address	Relayer	No
Sweep the remainder	Stealth address	No

- 01 **StealthAnnouncer indexes msg.sender**, so announcing from your own wallet would make the stealth address pointless. The relayer announces instead.
- 02 Funding arrives as a **Circle Gateway mint**: what Arc records is Circle's minter paying the box. In the old design, eight boxes out of eight in one real wallet could be tied back without a viewing key; re-measured now, **zero**. APS stays in the design, no longer as a patch for a leak.

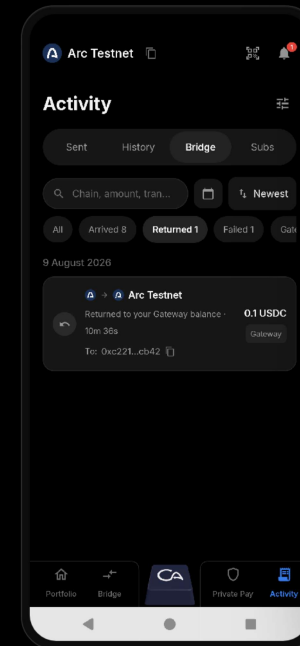
GETTING FUNDED

Bridging USDC in

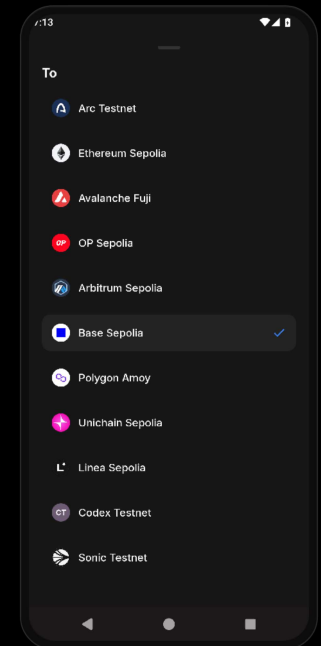
- 01 CCTP into Arc, five steps (price, approve, burn, attestation, mint), each one **linked to the explorer**.
- 02 **CCTP reaches twenty** testnet chains, **Gateway eleven**; the picker narrows to what the chosen engine really supports rather than offering a route that cannot run.
- 03 A failed mint **returns the money**: the row says returning, then returned. Measured twice, under ten minutes end to end, fee included.



Five steps, each one verifiable.



Returned is not failed: back in the Gateway balance in 10m 36s.



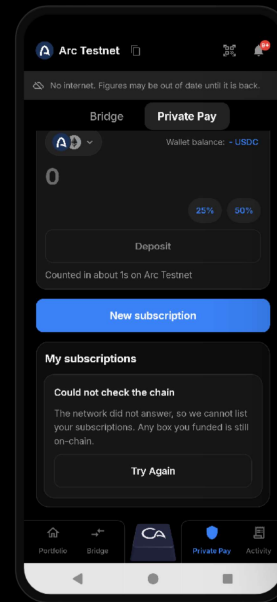
The chain names the app actually uses, with real logos.

PRODUCT PHILOSOPHY

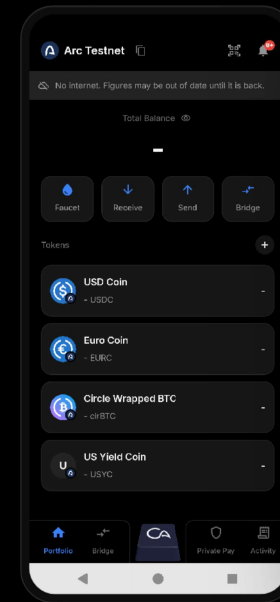
Failure, stated plainly

A payments app that rounds an unknown down to zero has taught the user to distrust it. Every unknown here is named.

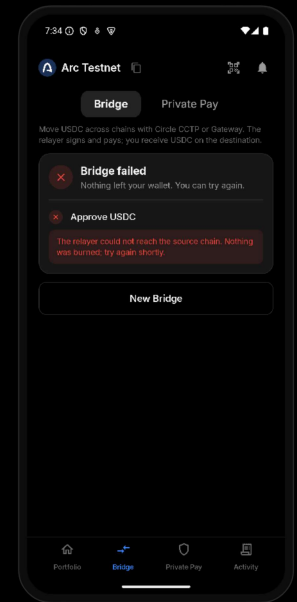
- 01 When the chain cannot be read it says **the network did not answer**, not "you have no subscriptions".
- 02 A figure that was never read is **a dash**. A figure that was read and has gone stale stays on screen with a line saying so. Neither one is ever a fake 0.
- 03 A failed bridge names the step, the reason, and that **nothing left your wallet**.



Unreadable, said out loud.



No internet: every figure a dash, and a line above the balance saying so.



Which step, and why.

WHY THIS CHAIN

Lock-then-claim needs two transactions, and Arc makes that economical

Gas

Gas is USDC

The second transaction is cheap and predictable, and there is no separate gas token to acquire before you can move your money.

Finality

Sub-second, deterministic

The transfer settles the moment the code is entered. The recipient never watches a spinner.

Approvals

Permit2, one signature

No separate approve transaction per send.

Liquidity

CCTP and Gateway wired

Both of Circle's cross-chain routes are a single tab in the app.

LANDSCAPE

Stopping the send, without an arbiter or custody

APPROACH	STOPS THE SEND	RECOVERABLE AFTER	NEEDS AN ARBITER	TAKES CUSTODY	PLAIN P2P
Wallet address-book warning	No	No	No	No	Yes
Poisoning detection service	Warns only	No	No	No	Yes
Commerce escrow	No	By dispute	Yes	Yes	No
Circle Refund Protocol	No	By mediator	Yes	Yes	No
Ctrl+ArcZ	Yes	By the sender	No	No	Yes

Circle's Refund Protocol solves a different problem on purpose: it is a commerce escrow built around an arbiter. Adding an arbiter here would break the one property that makes a protected-transfer contract worth trusting.

TRUST

What is proven

The contract is ownerless: no owner, no pause, no upgrade path.

124

Foundry tests on the contract

388

unit tests in the TypeScript SDK

390

unit tests in the native Android client, 47 classes

1081

tests across the repo: 124 Foundry, 388 SDK, 390 Android, 146 demo-kit, 33 keeper

0

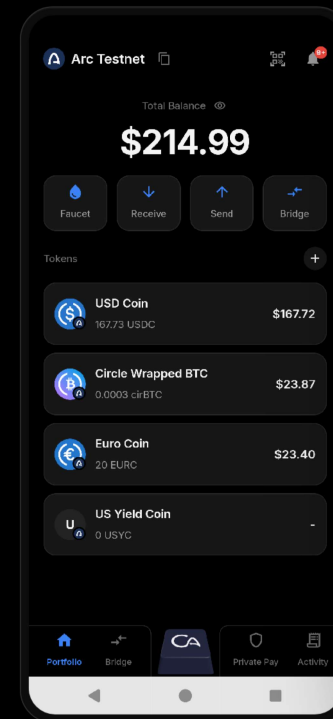
admin functions that can touch a locked transfer

live

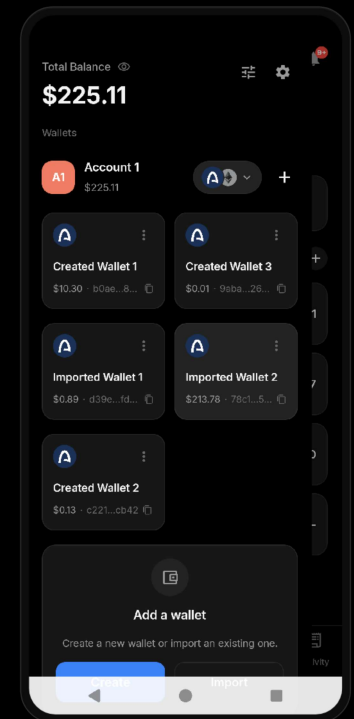
on Arc Testnet, testnet only, unaudited

CTRLARCZ

0x8dAb7148cdc31DAcad6d7e12161AA3DEdb572Dca



Every number on these screens was read from Arc Testnet.



Several wallets, created and imported.

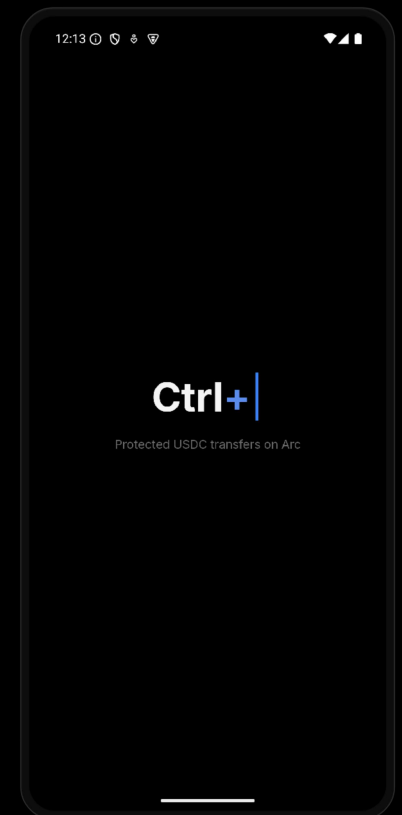
Ctrl+ArcZ

A wrong send should be refusable, and a right one should be reversible until it is claimed.

The undo shortcut, typed, erased and retyped as the product name.

ARC TESTNET · LIVE

0x8dAb...2Dca



Ctrl+Z becoming Ctrl+ArcZ.